

Secretary

Architecture and workflow

Company: **The President**

Agent: **Secretary**

A company operations assistant with bounded permissions. It summarizes and drafts. People remain responsible for decisions and external actions.

Product goal

Secretary gives The President one place to understand company activity across Slack, WhatsApp, Calendar, GitHub, and approved social accounts. It should preserve context, surface decisions and follow-ups, help coordinate work, prepare and publish approved social content, and produce a monthly report with links to its evidence.

Treat this as a company operations assistant with bounded permissions. It summarizes and drafts; people remain responsible for decisions and external actions. Every factual claim should point to an authorized source, and the system should show when it lacks access or coverage.

Initial scope and assumptions

- Begin with one company workspace and a small set of opted-in Slack channels, GitHub repositories, and calendars.
- Start read-only. Add message posting, calendar changes, and issue creation later as explicit, reviewable actions.
- Treat WhatsApp as a separate integration milestone. Use an official business integration and only conversations the organization is authorized to process. Personal accounts and private chats are out of scope.
- Treat social publishing as a separate, approval-gated workflow. Connect only company-owned or authorized accounts, and account for platform-specific content, media, and scheduling capabilities.
- Store normalized activity and report evidence with source references. Avoid retaining full message bodies longer than needed, and make retention configurable.
- Do not use activity counts as a proxy for individual performance. Reports focus on projects, decisions, outcomes, risks, and open follow-ups.

System architecture



Figure 1. Secretary system architecture.

Main components

Component	Responsibility
FastAPI API	Authentication, workspace configuration, chat and report endpoints, OAuth callbacks, and approval endpoints.
Connector adapters	Provider-specific OAuth, webhooks, pagination, rate-limit handling, and incremental sync. Keep provider APIs behind a common interface.
Background workers	Run syncs, webhook processing, scheduled summaries, report generation, and retries outside request handlers.
PostgreSQL	Store workspace settings, connector cursors, normalized events, conversations, tasks, approvals, reports, and audit records.
Search and retrieval	Find relevant authorized records for answers and reports. Start with PostgreSQL full-text search, and add a vector index only if evaluation shows it helps.
Assistant and workflow service	Select relevant evidence, call narrowly defined read tools, draft a response or action, and enforce approval rules.
Social publishing orchestrator	Manage a post from brief to platform-specific drafts, review, schedule, publish, and outcome tracking.
Model adapter	Keep prompts and provider-specific model calls behind an interface. Require structured output for extracted decisions, tasks, and report sections.
Audit log	Record who connected a source, what the agent accessed or proposed, who approved an action, and the result. Do not log secrets or unnecessary message content.

Suggested Python project layout

```
company-agent/
├─ pyproject.toml
├─ uv.lock
├─ README.md
├─ src/company_agent/
│   ├─ main.py           # FastAPI app and lifespan setup
│   ├─ api/              # Routers and request/response schemas
│   ├─ core/             # Settings, auth, permissions, logging
│   ├─ connectors/       # Slack, GitHub, Calendar, WhatsApp adapters
│   ├─ ingestion/       # Webhooks, sync cursors, normalization
│   ├─ assistant/       # Retrieval, tools, prompts, model adapter
│   ├─ social/          # Campaign orchestrator, platform adapters
│   ├─ workflows/       # Approvals, tasks, scheduling, reports
│   ├─ db/              # SQLAlchemy models, sessions, migrations
│   └─ jobs/            # Worker tasks and schedules
└─ migrations/
```

Figure 2. Project layout.

Use `uv` for environment and dependency management, for example `uv init`, `uv add fastapi`, and `uv run` followed by a command. Pin dependencies in `uv.lock`. Keep the API, worker, and scheduled jobs as separate processes that share the same Python package and database.

Data flow and core records

1. A workspace admin connects a provider through OAuth or a provider webhook and selects exactly which channels, repositories, calendars, and company social accounts are in scope.
2. A connector fetches events incrementally, converts them to a common event shape, and saves the provider ID, timestamp, source URL, access scope, and relevant content fields.
3. The assistant retrieves only records the requesting user is allowed to see. It returns summaries with source links and distinguishes direct evidence from interpretation.
4. A scheduled report job aggregates eligible events over a stated date range, prepares a draft, and sends it to a reviewer before distribution.

Core records should include `Workspace`, `User`, `Connector`, `SyncCursor`, `SourceEvent`, `Conversation`, `Task`, `Decision`, `Approval`, `Report`, and `AuditEvent`. Every stored record that came from a provider should carry a workspace ID and a source reference. Provider tokens belong in a secrets store or encrypted credential storage, never in source control or ordinary logs.

Workflows

1. Connect and configure a source

1. An admin chooses a provider and completes its authorization flow.
2. The app displays granted scopes and lets the admin select included channels, repositories, calendars, or business conversations.
3. The connector performs an initial sync, records a cursor, and reports what it could and could not access.
4. The admin can pause sync, change scope, disconnect, and request deletion of stored source data.

2. Keep activity current

1. Receive a provider webhook where available. Validate its signature and deduplicate by provider event ID.
2. Enqueue processing and acknowledge quickly. For missed events, periodic incremental sync resumes from the saved cursor.
3. Normalize events into common types such as `message`, `meeting`, `pull_request`, `release`, `issue`, `security_alert`, and `decision_candidate`.
4. Store source links, timestamps, author and team identity where permitted, and minimal useful text. Apply retention and deletion rules.
5. Retry transient provider failures with backoff, and surface persistent failures in connector health status.

3. Answer a team question

1. Authenticate the user and resolve workspace and source permissions.
2. Search relevant activity for the requested time range and topic. Filter before the model sees content.
3. Ask the model to produce an answer and a list of supporting event IDs in a constrained structure.
4. Verify that cited events were retrieved and remain accessible to that user.
5. Return a concise answer with links, date boundaries, and an explicit uncertainty or coverage note when appropriate.

4. Schedule a meeting (later phase)

1. The user asks for a meeting and provides or confirms attendees, duration, and purpose.
2. The agent reads availability only for calendars authorized for that request and proposes options.
3. The user selects an option and reviews the title, attendees, time zone, and description.
4. After explicit confirmation, the calendar adapter creates the event and returns the provider link. The audit record captures the request, approver, and result.

5. Produce a monthly report

1. A scheduled job selects the workspace's configured reporting window and eligible sources.

2. It groups evidence by projects or topics and drafts sections for shipped work, key decisions, meetings, open follow-ups, security alerts, and blockers.
3. Each claim links to evidence. The report states its coverage window, connected sources, excluded sources, and sync gaps.
4. A designated reviewer edits or approves the draft.
5. The approved report is delivered to the configured destination, and its final version and evidence references are retained according to policy.

6. Prepare and publish a social post

1. A team member gives the agent a campaign brief: goal, audience, key points, destination accounts, assets, and any required links or disclosures.
2. The orchestrator retrieves approved brand guidance and relevant company facts. It drafts platform-specific versions and checks length, required fields, media format, links, and configured review rules.
3. A reviewer sees each version, destination account, media asset, and proposed publish time together. The reviewer edits, rejects, or approves the exact content.
4. The scheduler queues approved posts. Before publishing, it checks that approval is still valid and that the destination account remains connected.
5. A platform adapter publishes or schedules the post according to that platform's supported API. It records the platform post ID, URL, status, timestamps, and any error. Retries must not create duplicate posts.
6. The team can pause or cancel queued posts where supported. The monthly report can summarize published content and available engagement metrics, with source links and metric coverage stated.

The orchestrator owns campaign state and approvals, and platform adapters own API differences. Do not assume every platform supports the same post formats, scheduling, deletion, or analytics. Model those as adapter capabilities and show unavailable actions in the review UI.

API shape (initial)

Representative endpoints:

Endpoint	Purpose
GET /health	Liveness and readiness status.
GET /v1/connectors POST /v1/connectors/{provider}/authorize	Connector setup.
POST /v1/webhooks/{provider}	Validated provider events.
POST /v1/ask	Ask a sourced question against authorized records.
GET /v1/reports GET /v1/reports/{report_id}	List and view reports.
POST /v1/reports/{report_id}/approve	Reviewer approval.
POST /v1/social/campaigns	Create a campaign draft from a brief.
POST /v1/social/posts/{post_id}/approve	Approve exact content and destination details.
GET /v1/social/posts POST /v1/social/posts/{post_id}/cancel	Inspect or cancel queued posts where supported.
GET /v1/approvals POST /v1/approvals/{approval_id}/decision	Review proposed actions.

Keep provider-specific payloads inside connector modules, and expose stable application-level schemas to the UI and assistant.

Security and reliability requirements

- Enforce workspace isolation and source-level permissions in the database and retrieval layer, not only in prompts.
- Request the minimum provider scopes needed. Verify webhook signatures, validate OAuth state, and rotate or revoke credentials on disconnect.
- Treat messages, issue text, and documents as untrusted input. They may contain instructions. Never let retrieved content expand tool permissions or bypass approval rules.
- Separate read tools from write tools. Require a human approval record for sending messages, creating calendar events, changing issues, or distributing reports.
- Require approval of the exact social copy, destination account, assets, and scheduled time. Edits after approval invalidate that approval. Keep campaign drafts distinct from published content.
- Make actions idempotent where possible and record provider response IDs for safe retries.
- Provide deletion and retention controls, encrypted transport and storage, access logs, backups, and a clear incident response owner before production data is connected.

- Never silently state that all company activity is covered. Show source coverage and sync health alongside reports.

Delivery plan

Phase 1: foundations

Create the FastAPI application with `uv`, PostgreSQL schema, workspace and user authentication, audit records, and a local development setup. Add sample fixture data so the experience can be built without live credentials.

Phase 2: read-only useful slice

Connect GitHub and one opted-in Slack workspace and channel set. Implement incremental sync, permission-filtered search, sourced question answering, and a draft monthly report. Add report review before delivery.

Phase 3: calendar and operational follow-ups

Add calendar read access, meeting preparation, and task and decision tracking. Measure whether extracted follow-ups are accurate and useful before enabling calendar writes.

Phase 4: approved actions and WhatsApp

Add narrowly scoped write actions behind a review queue, including the social publishing orchestrator and calendar and message actions. Integrate WhatsApp Business only after confirming the organization's account, consent, permitted conversations, and retention needs.

Phase 5: company-wide operation

Add additional teams and sources, admin controls, source-health dashboards, retention automation, report scheduling, and evaluation from user corrections and missed or incorrect summaries.

MVP acceptance criteria

- An admin can connect and scope at least GitHub and Slack sources, then disconnect them.
- The system syncs incrementally and shows last successful sync and failures.
- A user can ask a question and receive a useful answer with working source links, limited to authorized records.
- A monthly report draft covers a visible date range, cites its claims, and describes source gaps.
- A reviewer can approve or reject the report before delivery.
- No external write action occurs without an explicit user approval and an audit entry.